```
// ... $html5 heredoc as before
$dom->loadHTML($html5, LIBXML_NOERROR);
print "Document loaded." . PHP_EOL;
var_dump(libxml_get_errors());
```

```
Output:
Document loaded.
array(0) {
}
```

... but as you can see, you will then lose a lot of possibly important information. Parsing HTML5 with `DOMDocument` comes with a fairly hefty disclaimer: if the HTML is not well-formed, you will likely get a different DOM structure compared to the same markup parsed with `Dom\HTMLDocument`. The rules for handling things like missing closing tags differ between HTML4 and 5.

## Namespaces (Again!)

I still haven't fully addressed my decision to demonstrate `Dom\XPath`, and the answer to that lies with `Dom\HTMLDocument` and namespaces. Now, you'll recognize the word "namespace" because PHP namespaces were introduced in Chapter 3, "Functions," and were also explored in **Object Fundamentals**. Namespaces also exist in XML (and therefore HTML). They exist as a mechanism to allow elements with different contents and meaning to coexist in the same document (and avoid things like conflicts when combining XML fragments from different sources, just like we would use them to avoid naming conflicts when using classes from different libraries that share a name). They work by prepending element names (tags in the document) with a namespace name. A book `<project>` could become `<b:project>`, a coding project could become `<c:project>`, and so on. XML namespaces must be defined, which can be done with an `xmlns:<PREFIX>="URI"` attribute in the start tag of the first namespaced element:

```
<data>
  <c:project xmlns:c="http://some.domain/code_project">
    <c:codebase></c:codebase>
  </c:project>
```